

Inteligencia en Red: E1. Estrategias de búsqueda y sistemas de reglas

Diego de Andrés Casanova 100496145

Francisco Jimeno Fernández 100499080

1. Verificación del mapa (3 celdas + zona peligrosa)

E1

1.1) Cálculo de celdas.

Celda 1: (col=3; fila=9)

coste base: +1
penalización por borde: +3
Distancia euclídea al rival: (3,1)

$$d = \sqrt{(3-3)^2 + (9-1)^2} = \boxed{8} \rightarrow \text{ZONA CRÍTICA } d \leq 2$$

zona crítica: +8

Total: $1+3+8 = \boxed{12}$

Celda 2: (col=0; fila=7)

coste base: +1
penalización por borde: +3
Distancia euclídea

$$d = \sqrt{(3-0)^2 + (7-7)^2} = \boxed{3} \rightarrow \text{ZONA DE PRECAUCIÓN } 3 \leq d \leq 4$$

zona de precaución: +4

Total: $1+3+4 = \boxed{8}$

Celda 3: (col=1; fila=0)

coste base: +1
penalización por borde: +3
Distancia euclídea:

$$d = \sqrt{(3-1)^2 + (7-0)^2} = \boxed{7.28} \rightarrow \text{FUERA DE ZONAS DE ALERTA } d > 7$$

TOTAL: $1+3 = \boxed{4}$

Figura 1.

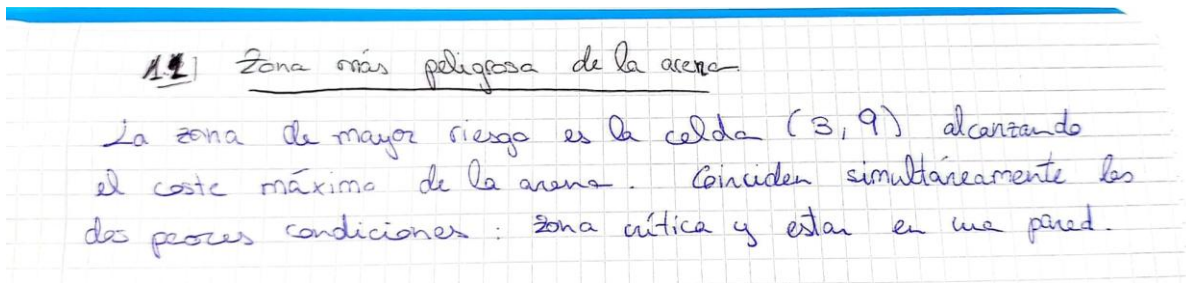


Figura 2.

2. Formulación del problema

-Estado ¿Cómo representáis una posición en el mapa?

Usamos un par ordenado que representa la posición del bot. (x,y)

-Estado inicial ¿Dónde empieza la búsqueda?

$(0,0)$

-Acciones ¿Cuáles son los movimientos posibles del bot?

En cualquier dirección de manera ortogonal mientras sea una celda adyacente y no salga de los límites de la arena.

-Prueba de objetivo ¿Cuándo ha llegado el bot al refugio?

Se cumple cuando las coordenadas del bot actuales coincidan con la celda $(8,3)$.

-Coste de cada acción ¿Cuánto cuesta moverse de una celda a otra?

Sera la suma total del coste base y las posibles penalizaciones añadidas. Por ejemplo si se encuentran a una distancia concreta del enemigo o están ubicados en el borde de la arena.

3. Búsqueda no informada

Hemos elegido el algoritmo de búsqueda de corte uniforme también conocido como algoritmo de Dijkstra

-¿Porqué este algoritmo y no otro?

Al compararlo con BFS que solo es óptimo cuando todos los caminos tienen el mismo coste o DFS que tampoco nos garantizaba ser óptimo nos pareció mejor idea usar UCS que expande siempre el nodo con menor coste acumulado y garantiza una opción óptima.

-¿Garantiza encontrar el camino de menor coste?

Sí, garantiza encontrar el camino óptimo debido a que todos los costes por celda son estrictamente positivos.

-¿Cuál es el punto débil del algoritmo en este problema concreto?

El punto débil del UCS es que expande los nodos concéntricamente en todas las direcciones basándose únicamente en el coste acumulado y por tanto explora muchas celdas que se alejan de la meta por falta de orientación hacia el objetivo.

-EJECUCIÓN

Paso	Nodo expandido (col, fila)	g(n)	Paso	Nodo expandido (col, fila)	g(n)
1	(0, 0)	0	19	(5, 2)	12
2	(0, 1)	4	20	(7, 1)	12
3	(1, 0)	4	21	(4, 0)	13
4	(1, 1)	5	22	(4, 3)	13
5	(2, 1)	6	23	(6, 2)	13
6	(1, 2)	7	24	(7, 2)	13
7	(2, 0)	8	25	(8, 1)	13
8	(2, 2)	8	26	(0, 3)	14
9	(3, 1)	8	27	(1, 4)	14
10	(0, 2)	9	28	(5, 0)	14
11	(1, 3)	9	29	(5, 3)	14
12	(4, 1)	9	30	(8, 2)	14
13	(2, 3)	10	31	(2, 4)	15
14	(3, 2)	10	32	(3, 3)	15
15	(5, 1)	10	33	(6, 0)	15
16	(4, 2)	11	34	(6, 3)	15
17	(6, 1)	11	35	(7, 3)	15
18	(3, 0)	12	36	(8, 3)	15

-Camino resultante

(0,0)->(0,1)->(1,1)->(2,1)->(3,1)->(4,1)->(5,1)->(6,1)->(7,1)->(7,2)->(8,2)->(8,3)

-Coste total

15

-Nodos expandidos

36

4. Búsqueda informada

Hemos elegido A* con heurística Manhattan.

$$h(\text{col, fila}) = |\text{col} - 8| + |\text{fila} - 3|$$

-¿Es admisible?

Sí, porque calcula la mínima distancia teórica con pasos ortogonales asumiendo el coste mínimo por celda el cual es el coste base = 1. En el mapa real las celdas tienen un coste igual o mayor que 1 por lo tanto el coste real restante jamás será inferior a h(n). Es consistente porque al pasar a una celda vecina la heurística disminuye como máximo en 1 unidad y el coste de transición es mayor o igual que 1.

-EJECUCIÓN

Paso	Nodo expandido (col, fila)	g(n)	h(n)	f(n)	Paso	Nodo expandido (col, fila)	g(n)	h(n)	f(n)
1	(0, 0)	0	11	11	9	(4, 1)	9	6	15
2	(0, 1)	4	10	14	10	(5, 1)	10	5	15
3	(1, 0)	4	10	14	11	(6, 1)	11	4	15
4	(1, 1)	5	9	14	12	(7, 1)	12	3	15
5	(2, 1)	6	8	14	13	(7, 2)	13	2	15
6	(1, 2)	7	8	15	14	(8, 1)	13	2	15
7	(2, 2)	8	7	15	15	(8, 2)	14	1	15
8	(3, 1)	8	7	15	16	(8, 3)	15	0	15

-Camino resultante

(0,0)->(0,1)->(1,1)->(2,1)->(3,1)->(4,1)->(5,1)->(6,1)->(7,1)->(7,2)->(8,2)->(8,3)

-Coste total

15

-Nodos expandidos

16

5. Comparación de algoritmos

-¿Los dos algoritmos encuentran el mismo camino? ¿Tienen el mismo coste? Si difieren, ¿cuál es mejor y por qué?

Sí, ambos encuentran el camino óptimo de coste total 15. Es el resultado de usar una heurística admisible y consistente en A*. Es mejor el algoritmo de búsqueda informada ya que expande menos nodos encontrando el camino óptimo igualmente

-¿Cuál ha explorado menos nodos? ¿Qué implica eso para un bot con límite de tiempo por turno?

A* ha explorado 16 nodos respecto a los 36 nodos de UCS lo cual es una reducción de nodos considerable. Expandir menos nodos reduce el consumo de tiempo de la CPU y por tanto es menos probable que pierdas el turno por timeout.

-El camino más directo sería ir por la fila 0 hasta la columna 8 y bajar a (8,3). Calculad su coste a mano. ¿Es peor respecto al camino de vuestro algoritmo informado?

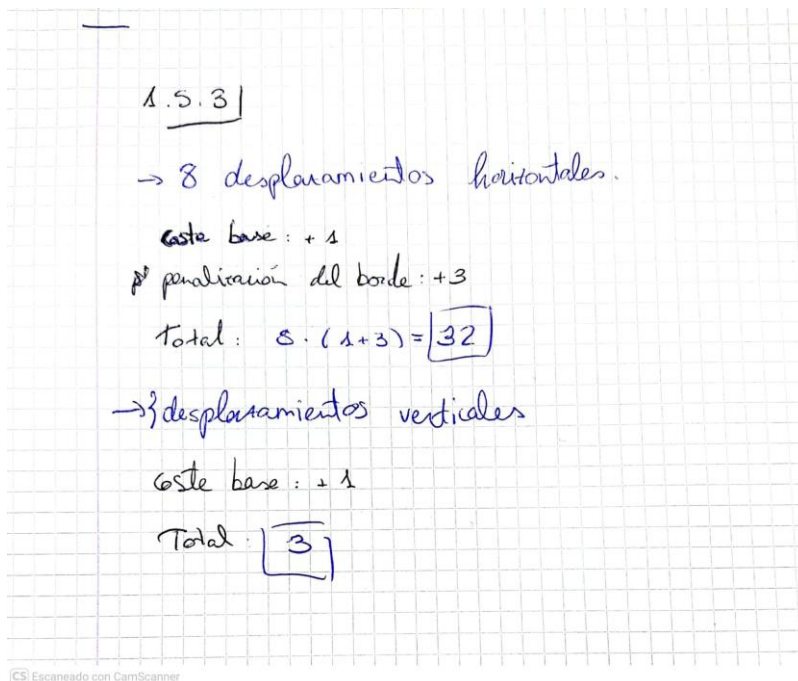


Figura 3.

El camino es bastante peor porque moverse pegado a las paredes se acumula mucha penalización.

-¿Tiene sentido implementar este sistema en el bot real? ¿Qué problema habría si el rival se mueve entre el momento en que se calcula la ruta y el momento en que se sigue?

El algoritmo asume un entorno estático si el rival se mueve el camino se vuelve obsoleto. La arena es continua y no esta dividida en una base simplificada de 10x10.

6. ¿Qué sabe el bot?

El sistema de reglas trabaja sobre hechos. Separamos los hechos en tres grupos según de dónde salen, y para cada uno decimos en qué reglas se usa y por qué hace falta.

6.1. Hechos siempre disponibles (estado propio)

Hecho	Reglas que lo usan	Por qué es relevante
self.energy	R1, R12	Es la vida del bot. Decide cuándo dejamos de combatir y nos retiramos, y cuánto podemos gastar en disparar.
self.gun_heat	R9 a R12	Si el cañón está caliente set_fire() no hace nada. Solo tiene sentido evaluar las reglas de disparo cuando vale 0.
self.turn_number	visto, R2, R6	Reloj del bot. Sirve para saber si un escaneo o un impacto son recientes y para alternar el sentido del zigzag.
self.x, self.y	R1, R4, R5, R6, R8	Nuestra posición. Hace falta para calcular hacia dónde girar (bearing_to) y la distancia al rival.
self.arena_width, self.arena_height	R1	Tamaño de la arena (800x600). Con él se calculan las esquinas candidatas a refugio.

6.2. Hechos que llegan por el radar (se guardan en on_scanned_bot)

Entre escaneos el bot no sabe dónde está el rival, así que on_scanned_bot solo guarda lo último que vio en variables y no toma ninguna decisión. Las decisiones se toman todas en run().

Hecho	Reglas que lo usan	Por qué es relevante
self._rival_x, self._rival_y	R1, R4, R5, R6, R8	Última posición conocida del rival. Con ella apuntamos el cañón, nos acercamos o alejamos y elegimos la esquina más lejana.
self._rival_dist	R4, R5, R6, R9, R10, R11	Distancia al rival. Es el hecho que decide si acercarse, alejarse o hacer zigzag, y con qué potencia disparar.
self._ultimo_escaneo	visto	Sirve para saber si la información del rival sigue siendo válida.

No usamos la energía, la velocidad ni la dirección del rival. Las dos últimas las probamos para predecir el tiro y las quitamos con datos, la energía se registra en el CSV y la analizaremos en E2.

6.3. Hechos que llegan por eventos

Evento	Hecho que guarda	Regla	Por qué es relevante
on_hit_by_bullet	self._turno_impacto, self._dir_bala	R2	Nos han dado. Si seguimos en la misma línea nos volverán a dar, así que hay que cambiar de trayectoria.
on_hit_wall	self._choque_pared	R3	Estamos empujando contra la pared

Además el bot guarda una memoria propia, self._sentido (1 = adelante, -1 = atrás), porque el rebote en la pared y el zigzag necesitan recordar en qué sentido íbamos. Todos estos hechos se reinician en on_round_started para no arrastrar datos de la ronda anterior.

6.4. Condición de "visto" y elección de N

visto = (self._rival_dist is not None and self.turn_number - self._ultimo_escaneo < N), con N = 8.

- **Por qué 8:** El radar gira 45° por turno, así que da una vuelta completa cada $360 / 45 = 8$ turnos. En un 1 contra 1, pase lo que pase, el rival vuelve a aparecer en el radar como muy tarde 8 turnos después del último escaneo. Con $N = 8$ la información nunca caduca entre dos barridos consecutivos.
- **Por qué no otro valor:** Con N pequeño (1 ó 2) el bot casi nunca dispararía, porque solo tendría al rival "visto" el turno del escaneo. Con N grande (20 o 30) seguiríamos apuntando a una posición vieja.
- **Coste de $N = 8$:** En 8 turnos el rival se mueve como máximo 64 px. A las distancias medias en las que disparamos (150–400 px) es un error asumible, y si el rival muere o se pierde, a los 8 turnos pasamos automáticamente a la regla por defecto.

6.5. El radar: barrer o seguir

El radar no es una regla de decisión, es la fuente de hechos, pero cómo lo movemos cambia mucho la calidad de esos hechos. El esqueleto del enunciado barre 45° siempre; nosotros lo hacemos en dos modos:

- **Buscar:** Si no tenemos rival "visto": barrer 45° por turno (vuelta completa cada 8 turnos, de ahí $N = 8$).
- **Seguir:** Si lo tenemos visto: girar el radar hacia su última posición más 20° de margen, alternando el lado. Así el haz pasa por encima del rival todos los turnos y lo volvemos a escanear cada turno, en vez de cada 8. Si el rival se sale del haz, a los 8 turnos visto pasa a falso y volvemos a barrer.

La consecuencia es que en combate los hechos del rival tienen normalmente 0 ó 1 turno de antigüedad, y $N = 8$ queda como red de seguridad para el caso peor.

7. Tabla de reglas

El sistema tiene 12 reglas en dos grupos. En el grupo de movimiento solo gana una regla por turno (if/elif en orden de prioridad). En el grupo del cañón también gana una sola regla de disparo, pero el grupo entero se evalúa en el mismo turno que el movimiento, porque en Robocode moverse y disparar son acciones simultáneas. El radar (apartado 6.5) no es una regla: es la fuente de hechos.

7.1. Grupo movimiento (una sola regla gana)

Nº	SI (condición)	ENTONCES (acción)	Prio.	Propósito
R1	energía < 20	ir hacia la esquina más alejada del rival (con 100 px de margen a la pared), marcha atrás si queda a la espalda; una vez a menos de 60 px del refugio, zigzag perpendicular al rival	1	Retirada: dejar de combatir cuando un impacto más nos deja casi muertos.
R2	nos ha dado una bala hace ≤ 1 turno	invertir el sentido, girar 90° respecto a la dirección de la bala y avanzar	2	Esquiva: salir de la línea de tiro y no volver por donde el rival ya nos ha acertado.
R3	hemos chocado con la pared o, siguiendo así, en 80 px nos salimos del margen de 50 px	invertir el sentido, girar 30° y avanzar	3	Rebote preventivo: no llegar a la pared ni quedarnos empujando contra ella.
R4	visto y distancia > 400	ir hacia el rival (marcha atrás si queda a la espalda)	4	Acercarse: a más de 400 px casi no acertamos.
R5	visto y distancia < 150	ir en dirección contraria al rival (marcha atrás si lo tenemos delante)	5	Alejarse: evitar la embestida y los disparos.
R6	visto y $150 \leq \text{distancia} \leq 400$	ponerse perpendicular al rival y avanzar; cambiar de sentido cada 8–30 turnos (intervalo al azar)	6	Zigzag lateral: mantener la distancia buena siendo un blanco difícil.
R7	en otro caso (no visto)	girar 30° y avanzar en el sentido actual	7	Regla por defecto: patrullar en curva para que el radar encuentre al rival.

7.2. Grupo cañón (se ejecuta además del movimiento)

Nº	SI (condición)	ENTONCES (acción)	Prio.	Propósito
R8	visto	girar el cañón hacia la última posición conocida del rival	1	Apuntar. No gasta energía ni depende del calor, así que se hace siempre que hay objetivo.
R12	visto, cañón frío y alineado, energía < 20	disparar con potencia 1	2	En retirada solo apostamos lo mínimo (ver apartado 8).
R9	visto, cañón frío y alineado, distancia < 150	disparar con potencia 3	3	De cerca casi seguro que acertamos: máximo daño.
R10	visto, cañón frío y alineado, $150 \leq \text{distancia} \leq 400$	disparar con potencia 2	4	Equilibrio entre daño, velocidad de la bala y coste.
R11	visto, cañón frío y alineado, distancia > 400	disparar con potencia 1	5	De lejos, bala rápida y barata: si fallamos perdemos poco.

"Cañón frío" es `gun_heat == 0` y "alineado" es que al cañón le falte menos que medio ancho de bot para apuntar al punto previsto. Si no se cumple alguna de las dos, no se activa ninguna regla de disparo: el bot apunta (R8) y espera al turno siguiente. Así no gastamos energía en balas que salen mientras el cañón todavía está girando.

7.3. Justificación de los umbrales

Umbral	Valor	Justificación
X (retirada)	20	Una bala de potencia 3 quita 16. Con menos de 20 de energía, un impacto fuerte más nos deja con 4 o menos, es decir, prácticamente fuera. Por encima de 20 aún aguantamos dos impactos medios ($10 + 10$), así que compensa seguir combatiendo. Equivale al 20 % de la vida inicial.
Distancia "cerca"	150	A esa distancia una bala de potencia 3 (11 px/turno) llega en unos 14 turnos y el cono de tiro es tan ancho que acertamos casi seguro: por eso potencia 3. Lo malo es que al rival le pasa lo mismo y además hay riesgo de embestida, por eso en movimiento nos alejamos.
Distancia "lejos"	400	Es la mitad del ancho de la arena. A 400 px una bala de potencia 1 tarda $400 / 17 \approx 24$ turnos, tiempo en el que el rival puede moverse hasta 190 px: acertar es cuestión de suerte. Por eso de lejos nos acercamos y solo tiramos balas de potencia 1, que cuestan 1 y van más rápido.
Potencias 3 / 2 / 1	según distancia	A menos distancia más probabilidad de acierto, así que apostamos más: potencia 3 (coste 3, daño 16) de cerca; potencia 2 (coste 2, daño 10) en la zona media; potencia 1 (coste 1, daño 4) de lejos. Todas están en el rango válido 0,1–3.
Cañón frío	<code>gun_heat == 0</code>	<code>set_fire()</code> no dispara si hay calor, así que evaluar las reglas de disparo antes sería inútil. <code>gun_heat</code> va de 0 a $\approx 1,6$.
Cañón alineado	$< \text{atan}(18 / \text{distancia})$, mínimo 3º	18 px es medio ancho de bot: si el error del cañón es menor que ese ángulo, la bala entra dentro del rival aunque no esté perfectamente centrada. A 150 px son 6,8º y a 400 px 2,6º; el mínimo de 3º evita que de lejos nunca lleguemos a disparar. El cañón gira hasta 20º por turno, así que normalmente se alinea en 1 ó 2 turnos.
Radio del refugio	60 px	Al llegar al refugio el bot no se para (un bot quieto se acierta siempre) sino que hace zigzag alrededor del punto. Así que por pruebas llegamos a 60 px
Radar extra	20º	Al seguir al rival, el radar se pasa 20º de largo alternando el lado. Así el haz barre unos 40º por encima del rival cada turno y lo re-escaneamos aunque se mueva.
Esquiva	≤ 1 turno, 90º	La regla se activa solo el turno del impacto y el siguiente para que no bloquee el resto de reglas. Girar 90º es la forma más rápida de salir de la línea de la bala.
Zigzag	8–30 turnos, al azar	Hay dos formas de apuntarnos: a donde estamos y a donde estaremos si seguimos rectos. Con un periodo fijo de 10 turnos íbamos y volvíamos al mismo sitio justo en lo que tarda una bala (~ 18 turnos), y con los CSV comprobamos que un rival que disparase a "donde estamos" nos habría acertado el 60–75 % de las veces. Con tramos largos nos acierta el que predice en línea recta. Un intervalo al azar entre 8 turnos (~ 64 px) y 30 (~ 240 px) no le da un patrón a ninguno de los dos.
Margen de pared / anticipo	50 px / 80 px	Miramos 80 px por delante (10 turnos a velocidad máxima) y si ese punto queda a menos de 50 px de una pared, invertimos antes de llegar. Así evitamos el daño del

		choque y no nos quedamos pegados al borde, que en el mapa del apartado 1 es la zona con +3 de coste.
Marcha atrás	si el rumbo está a más de 90º	El cuerpo gira como mucho 10º por turno: dar la vuelta entera lleva unos 18 turnos avanzando en arco hacia donde no queremos (en la primera versión eso provocaba embestidas al intentar alejarnos). Si el destino queda a la espalda, giramos lo justo y vamos marcha atrás.
Margen refugio	100 px	En el mapa del apartado 1 las celdas del borde cuestan +3 porque no hay escapatoria lateral. Por eso el refugio no es la esquina exacta sino un punto a 100 px (≈ 3 anchos de bot) de las dos paredes.
N (visto)	8	Una vuelta completa del radar (ver apartado 6.4).

7.4. Lo que probamos y quitamos: la predicción del tiro

La primera versión de R8 apuntaba a donde iba a estar el rival suponiendo que seguía recto a su velocidad durante el vuelo de la bala (predicción lineal, usando e.speed y e.direction del escaneo). Lo comprobamos con los CSV de dos batallas de 10 rondas: para cada disparo miramos dónde estaba de verdad el rival cuando llegó la bala y recalculamos el error lateral que habría tenido la bala con distintas cantidades de adelanto.

Adelanto aplicado	Bala dentro del bot (< 18 px)	Error lateral mediano
0 (apuntar a donde está)	25 %	41 px
0,5 (mitad de la predicción)	23 %	42 px
1 (predicción lineal completa)	13 %	56 px

La predicción lineal acertaba la mitad que apuntar directamente. La explicación es que los bots de la arena no van rectos: oscilan alrededor de un punto (avanzan y retroceden, o hacen zigzag), y en los ~ 18 turnos que tarda una bala en recorrer 250 px el rival ya ha dado la vuelta. Extrapolar en línea recta lo coloca donde nunca va a estar. Por eso R8 apunta a la última posición conocida, que además es la regla más simple. Con más datos (E2) se podría estudiar una predicción que tenga en cuenta la oscilación.

8. Resolución de conflictos y completitud

8.1. Estrategia de resolución de conflictos

Usamos prioridad fija: las reglas de cada grupo están ordenadas y se aplica la primera cuya condición se cumple. En el código esto es un if/elif/else, así que el orden de los if ES la prioridad.

- **Por qué prioridad y no otra:** Es determinista y cuesta tiempo constante, que es justo lo que necesita un bot con límite de tiempo por turno.
- La especificidad no sirve aquí porque hay reglas igual de específicas que deben ganar siempre (la retirada tiene una sola condición y aun así debe estar por encima de todas).
- La regla más reciente tampoco tiene sentido: los hechos se actualizan cada turno y varias reglas se activarían "a la vez".

El orden elegido va de lo urgente a lo rutinario: primero la supervivencia (R1), luego reaccionar a lo que nos acaba de pasar (R2 impacto, R3 pared), después la táctica según la distancia (R4, R5, R6) y por último la patrulla (R7). Un ejemplo: si nos dan una bala mientras estamos a más de 400 px, se activan R2 y R4; gana R2 porque salir de la línea de tiro es más urgente que acercarse; al turno siguiente R2 deja de cumplirse y volvemos a R4.

8.2. El caso "disparar" frente a "retirarse"

Situación: energía < 20, rival a menos de 150 px y cañón frío. En el grupo de movimiento gana R1 (retirarse), y en el grupo del cañón se cumplirían R9 (potencia 3) y R12 (potencia 1). Como set_forward y set_fire se ejecutan en el mismo turno, el conflicto no es físico sino de energía: ¿cuánto merece la pena apostar en un disparo cuando nos quedan menos de 20 puntos?

Opción	Si acierta	Si falla	Con 3 fallos seguidos
No disparar	—	0	0 (pero tampoco hacemos daño)
Potencia 3 (R9)	-3 + 9 devueltos = +6 de energía y 16 de daño al rival	-3 (más del 15 % de lo que nos queda)	-9: con 19 de energía pasamos a 10, un solo impacto medio nos mata
Potencia 1 (R12)	-1 + 3 devueltos = +2 de energía y 4 de daño al rival	-1 (un 5 %)	-3: con 19 pasamos a 16, seguimos aguantando un impacto de potencia 3

Las dos potencias compensan en media si acertamos más de 1 de cada 3 veces (cuesta p y devuelve 3·p), pero con poca energía lo que importa no es la media sino no morir en una racha mala. Por eso nuestra decisión es la regla R12: nos retiramos y a la vez disparamos con potencia 1. Retirarnos no nos cuesta nada porque la bala sale igual mientras nos movemos, y la potencia 1 es la apuesta que peor puede salir sin comprometer la retirada. Descartamos no disparar porque el rival está cerca y el cañón frío: es la situación en la que más probable es acertar y recuperar energía. Descartamos la potencia 3 porque tres fallos, que a un rival en movimiento le pueden pasar a cualquiera, nos dejan a un impacto de la derrota.

8.3. Completitud del sistema

Para cualquier situación posible hay al menos una regla aplicable. Lo comprobamos recorriendo todos los casos:

Situación	Grupo movimiento	Grupo cañón
energía < 20	R1 (retirada)	R8 si visto; R12 si además cañón frío y alineado
energía ≥ 20 y impacto hace ≤ 1 turno	R2 (esquiva)	R8 + R9/R10/R11 si visto, cañón frío y alineado
energía ≥ 20, sin impacto, choque con pared	R3 (rebote)	ídem
... visto y distancia > 400	R4 (acercarse)	R8 + R11 si cañón frío y alineado
... visto y distancia < 150	R5 (alejarse)	R8 + R9 si cañón frío y alineado
... visto y 150 ≤ distancia ≤ 400	R6 (zigzag)	R8 + R10 si cañón frío y alineado
... no visto	R7 (por defecto)	ninguna: no hay objetivo, el radar vuelve a barrer

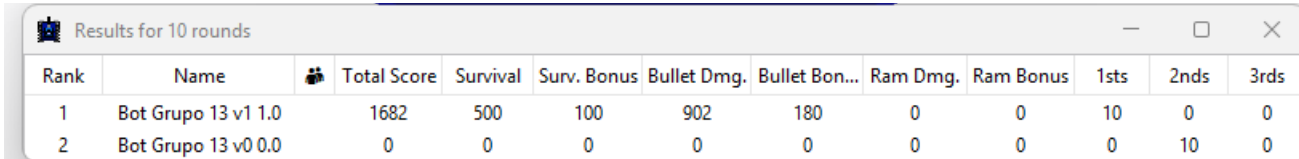
- **Movimiento:** La distancia es un número real y los tres intervalos (< 150, 150–400, > 400) cubren toda la recta sin solaparse. "Visto" es verdadero o falso. Así que dentro del grupo de movimiento siempre se entra en un if, y si no se entra en ninguno está el else de R7, la regla por defecto que exige el enunciado.
- **Cañón:** Las cuatro reglas de disparo cubren todas las distancias y el caso de poca energía. Cuando no se ve al rival, el cañón está caliente o todavía no está alineado no se dispara, y eso es una decisión, no un hueco: no hay nada a lo que disparar, el disparo no saldría o saldría desviado. En esos turnos el bot sigue haciendo algo (moverse, apuntar y mover el radar).
- **Radar:** Se mueve todos los turnos, fuera de las reglas: sigue al rival si lo tiene y barre si no, así que el bot nunca se queda sin percibir.

Como cada turno termina con un único go(), el bot siempre envía una orden por turno: nunca se queda bloqueado esperando.

8.4. Cómo está implementado en el código

- **Hechos:** `on_scanned_bot`, `on_hit_by_bullet` y `on_hit_wall` solo actualizan hechos (variables `self._...`). No deciden nada.
- **Motor:** `run()` es el motor de inferencias: cada turno calcula visto, evalúa el grupo de movimiento con `if/elif/else`, después el grupo del cañón (elige la potencia con R9–R12, apunta con R8 y dispara si está frío y alineado), mueve el radar y llama a `go()`. Cada bloque lleva su comentario # R1 ... # R12.
- **Base de conocimiento:** Los umbrales están como constantes al principio del fichero (`N`, `X_RETIRADA`, `D_CERCA`, `D_LEJOS...`) para poder ajustarlos en E2 con los datos sin tocar las reglas.
- **Acciones:** Solo se usan las versiones `set_` (no bloqueantes) y un único `go()` por turno, como recomienda el esqueleto.

9. Evaluación v0 vs v1:



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bon...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Bot Grupo 13 v1 1.0	1682	500	100	902	180	0	0	10	0	0
2	Bot Grupo 13 v0 0.0	0	0	0	0	0	0	0	0	10	0

Figura 4.

Se puede apreciar claramente una victoria arrolladora del bot v1 dando a entender que los cambios hechos sobre v1 han sido muy beneficiosos para la capacidad de combate del bot. Con una acumulación de 1682 puntos respecto a 0. 0 daños por embestidas dejando clara la superioridad táctica de v1 y su capacidad para ejercer daño desde la distancia. El bonus por supervivencia debido a que v0 no le ha causado prácticamente daño. A nuestro parecer las mejoras cumplen sobresalientemente con el objetivo de la práctica.

10. Declaración de uso de IA

Solamente nos hemos apoyado de herramientas de IA en este caso Claude para la depuración del código y análisis de las batallas pero tanto la decisión de las normas y la redacción de la memoria han sido hechas por nosotros en su totalidad.

Francisco Jimeno, Diego de Andrés